CS4780/5780 Homework 6

Songyu Ye

November 29, 2023

Problem 1: Kernel Width

The RBF (Gaussian) Kernel $K(x,z) = e^{\frac{-\|x-z\|^2}{2\sigma^2}}$, the Exponential Kernel, $K(x,z) = e^{\frac{-\|x-z\|}{2\sigma^2}}$, and the Laplacian Kernel, $K(x,z) = e^{\frac{-\|x-z\|}{\sigma}}$ all contain a length scale (σ).

Consider a situation where we do kernel regression on N data points in d dimensions. Assume that each data point is normalized, e.g. $\|x\| = 1$.

a. What would the interpolant look like if σ was very very small?

b. If we perturb the labels of one of the points, are the labels of test points very close to that perturbed point going to be more sensitive if sigma is large or small? Why?

c. Now imagine that, instead of having access to the *distance* between any two points x, z, you only have access to the inner product of the points $x^T z$. Re-write each of the kernel functions above in terms of $x^T z$, i.e, $||x - z||$ should not appear anywhere in your answer.

d. How should we set sigma for each kernel (Gaussian, Exponential, Laplacian) if the only data which has a positive inner product with the test data point should have a value $K(x,z) > .01$?

Solution:

1. If we consider the kernel function at small σ, we see that it is quite similar to a normal distribution, and so in particular If we have small σ we see that the interpolant is very sharp.

2. If σ is small, then the distribution has really low variance and so perturbing a point from the mean to something else can have a really large impact. More precisely, one can also look at the equations and note when we have a small σ, little changes between x and z in the kernel can lead to huge changes in your actual value.

3. Observe that

$$\begin{aligned} ||x - z||^2 &= (x - z)^T (x - z) \\ &= x^T x - 2x^T x + z^T z \end{aligned}$$

and so we can rewrite the kernels as

$$K(x,z) = e^{\frac{-(x^Tx-2x^Tz+z^Tz)}{2\sigma^2}}$$
$$K(x,z) = e^{\frac{-\sqrt{x^Tx-2x^Tz+z^Tz}}{2\sigma^2}}$$
$$K(x,z) = e^{\frac{-\sqrt{x^Tx-2x^Tz+z^Tz}}{\sigma}}$$

4. We are asking that $K(x,z) = .01$ should imply that $x^Tz = 0$. For the exponential kernel, we can rewrite this as

$$\begin{aligned}
e^{\frac{-\sqrt{x^Tx-2x^Tz+z^Tz}}{2\sigma^2}} &= .01 \\
\frac{-\sqrt{x^Tx-2x^Tz+z^Tz}}{2\sigma^2} &= \ln(.01) \\
\sqrt{x^Tx-2x^Tz+z^Tz} &= -2\sigma^2\ln(.01) \\
x^Tx-2x^Tz+z^Tz &= 4\sigma^4\ln^2(.01) \\
2 = 1-0+1 &= 4\sigma^4\ln^2(.01) \\
\sigma^4 &= \frac{2}{4\ln^2(.01)} \\
\sigma &= \sqrt[4]{\frac{2}{4\ln^2(.01)}} = 0.3918498
\end{aligned}$$

For the Gaussian kernel, we can rewrite this as

$$\begin{aligned}
e^{\frac{-(x^Tx-2x^Tz+z^Tz)}{2\sigma^2}} &= .01 \\
\frac{-(x^Tx-2x^Tz+z^Tz)}{2\sigma^2} &= \ln(.01) \\
x^Tx-2x^Tz+z^Tz &= -2\sigma^2\ln(.01) \\
2 = 1-0+1 &= -2\sigma^2\ln(.01) \\
\sigma^2 &= \frac{-2}{2\ln(.01)} \\
\sigma &= \sqrt{\frac{-2}{2\ln(.01)}} = 0.4659906
\end{aligned}$$

For the Laplacian kernel, we can rewrite this as

$$
\begin{aligned}
e^{\frac{-\sqrt{x^Tx-2x^Tz+z^Tz}}{\sigma}} &= .01 \\
\frac{-\sqrt{x^Tx-2x^Tz+z^Tz}}{\sigma} &= \ln(.01) \\
\sqrt{x^Tx-2x^Tz+z^Tz} &= -\sigma\ln(.01) \\
x^Tx-2x^Tz+z^Tz &= \sigma^2\ln^2(.01) \\
2 = 1-0+1 &= \sigma^2\ln^2(.01) \\
\sigma^2 &= \frac{2}{\ln^2(.01)} \\
\sigma &= \sqrt{\frac{2}{\ln^2(.01)}} = 0.3070926
\end{aligned}
$$

Problem 2: Constructing Kernels

In class, we have shown how to construct new kernels from existing valid kernels, following a few "kernel-preserving" rules. In this problem, we will prove that these rules indeed produce valid kernels. In order to justify that a function $k : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$ is a valid kernel function, we can show that either of the following properties hold:

1. The matrix $\mathbf{K}_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$ is **symmetric and positive semidefinite** for any set of vectors $\mathbf{x}_1, ..., \mathbf{x}_n \in \mathbb{R}^d$. Recall that a matrix $\mathbf{A}$ is positive semidefinite if $\mathbf{q}^T\mathbf{A}\mathbf{q} \geq 0$ for all vectors $\mathbf{q}$.
2. $k(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T\phi(\mathbf{x}_j)$ for some transformation $\phi : \mathbb{R}^d \to \mathbb{R}^m$.

Suppose k_1, k_2 are valid kernel functions, that is, their associated kernel matrices are **symmetric and positive semidefinite**. Show that the following kernels are valid:

(a) $k(\mathbf{x}_1, \mathbf{x}_2) = ck_1(\mathbf{x}_1, \mathbf{x}_2)$ for any $c \geq 0$.

(b) $k(\mathbf{x}_1, \mathbf{x}_2) = k_1(\mathbf{x}_1, \mathbf{x}_2)k_2(\mathbf{x}_1, \mathbf{x}_2)$.

(c) $k(\mathbf{x}_1, \mathbf{x}_2) = g(k_1(\mathbf{x}_1, \mathbf{x}_2))$ for a polynomial function g with positive coefficients.

Hints:

- When using the first condition, you need to prove **both** symmetry and positive semidefiniteness of the the kernel matrix.
- for part (c), you may assume that k_1 can be represented as an inner product of a finite-dimensional feature map, i.e. $k_1(\mathbf{x}_1, \mathbf{x}_2) = \phi_1(\mathbf{x}_1)^T\phi_1(\mathbf{x}_2)$.

Solution:

1. Scaling by a nonnegative constant certainly preserves the symmetry of our matrix, i.e. $(cK)^T = cK^T$. Moreover

$$q^T(cA)q = cq^TAq \geq 0$$

   since $c \geq 0$.

2. We have k_1 and k_2 are given by positive semidefinite symmetric matrices. Then we consider the product of the kernels $k(x_1, x_2) = k_1(x_1, x_2)k_2(x_1, x_2)$. Linear algebra tells us that we can extract the coefficients of the matrix which represents by plugging in the standard basis vectors e_i, e_j, in particular this will extract the ij coefficient of the matrix we are interested in. If we do this we in fact find out that $k_{ij} = k_{1ij}k_{2ij}$, which is the product of two nonnegative numbers and thus nonnegative.

   Therefore the matrix which corresponds to k is in fact given by the entrywise product of the two matrices coming from k_1 and k_2. This is clearly symmetric and positive semidefinite since it is the entrywise product of two such matrices.

3. We work with the second condition that k_1 can be represented as an inner product of a finite dimensional feature map. In particular, we have that $k_1(x_1, x_2) = \phi_1(x_1)^T\phi_1(x_2)$. Then we consider the function $k(x_1, x_2) = g(k_1(x_1, x_2))$ where g is a polynomial with positive coefficients. We see that this kernel will be represented by the inner product of the map

$g(\phi_1)$. Then we get that $k(x_1, x_2) = g(\phi_1(x_1))^T g(\phi_1(x_2))$ which is the inner product of a finite dimensional feature map and so k is indeed a kernel.

Problem 3: Kernelized Linear Regression

Consider you have the following dataset (with x values being scalar values and y values being labels)

No.	x	y
1	$x_1 = 1$	$y_1 = 0$
2	$x_2 = -1$	$y_2 = 2$

You are going to use a kernelized linear regression model with the linear kernel $k(x, x') = (x \cdot x' - 1)$. Given training data x_i, the kernelized linear regression model makes a prediction at a point x_* using

$$h(x_*) = \sum_i k(x_*, x_i)\alpha_i, \tag{1}$$

where $\boldsymbol{\alpha} = \mathbf{K}^{-1}\mathbf{y}$ is an n dimensional vector, $K_{ij} = k(x_i, x_j)$ and $\mathbf{y} = [y_1, \ldots, y_n]$.

(a) Write down the kernel matrix $\mathbf{K}$ of the model for the given dataset.

(b) For kernelized ridge regression the computation of the coefficients $\boldsymbol{\alpha}$ is changed to $\boldsymbol{\alpha} = (\mathbf{K} + \tau^2\mathbf{I})^{-1}\mathbf{y}$, where $\tau > 0$. Name an advantage that this could have over the model with $\tau = 0$.

(c) You are given the following test points:

No.	x_*	y_*
1	$x_1 = 0$	$y_1 = 1$
2	$x_2 = 2$	$y_2 = 1$

Let $\tau = 0$. What are the predictions of the model in Equation (1) on the test points? The model might not be "correct", so the predictions might not match the label. (Hint: You may find np.linalg.inv useful)

Solution:

1. Observe that

$$k(x1, x1) = k(x2, x2) = 0$$
$$k(x1, x2) = k(x2, x1) = -2$$

and so

$$K = \begin{bmatrix} 0 & -2 \\ -2 & 0 \end{bmatrix}$$

2. One reason that $\tau > 0$ could be an advantage is that it helps guarantee that your matrix is invertible. It also helps with overfitting (cause of the regularization constant)

3. We have $\alpha = K^{-1}y = \frac{-1}{4}\begin{bmatrix} 0 & 2 \\ 2 & 0 \end{bmatrix}\begin{bmatrix} 0 \\ 2 \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \end{bmatrix}$ from which we compute

$$h(x_1) = k(x_1, x_1)\alpha_1 + k(x_1, x_2)\alpha_2 = -1 \cdot -1 + -2 \cdot 0 = 1$$
$$h(x_2) = k(x_2, x_1)\alpha_1 + k(x_2, x_2)\alpha_2 = 1 \cdot -1 + -3 \cdot 0 = -1$$

Problem 1: CART

Given dataset below you want to build a regression tree, where x1 and x2 are the features and y is the label.

x1	x2	y
-2	8	positive
3	4	negative
12	6	positive
2	3	negative
8	-4	positive
6	3	positive

(a) Which feature(x1 or x2) would you use as your first split on your decision tree, why?

(b) Draw the regression **tree** built by the ID3-Algorithm introduced in class

(c) If we add a new categorical feature x3 as seen below, how would you account the new change? (Hint: think about what it means for the algorithm to "split" on a categorical feature.)

x1	x2	x3	y
-2	8	B	positive
3	4	R	negative
12	6	G	positive
2	3	R	negative
8	-4	B	positive
6	3	G	positive

Solution:

1. I would use feature x1 as my first split because it corresponds to the split with the lowest weighted entropy. We can compute this by looking at the two features. We have to compute some weighted entropies and figure out the one that gets us the minimum. We will only consider the computations for the obvious splits that have a chance at getting us the minimum. For x1, we only consider the split betewen 3 and 6 as it is obvious that this is the best split for the $x1$ feature.

$$H(x1) = 1/2 * (-\frac{1}{3}\ln(\frac{1}{3}) - \frac{2}{3}\ln(\frac{2}{3})) = 0.318257$$

For $x2$ we see that there are two interesting splits to consider, namely the split between 3 and 4 and the split between 4 and 6. We compute both of these:

$$H(x2)_1 = 2/3 * (-\frac{1}{2}\ln(\frac{1}{2}) - \frac{1}{2}\ln(\frac{1}{2})) = 0.5776$$

$$H(x2)_2 = 1/2 * (-\frac{1}{3}\ln(\frac{1}{3}) - \frac{2}{3}\ln(\frac{2}{3}) + 1/2 * (-\frac{1}{3}\ln(\frac{1}{3})) - \frac{2}{3}\ln(\frac{2}{3})) = 0.5449$$

so we see that the minimum is achieved by splitting on x1.

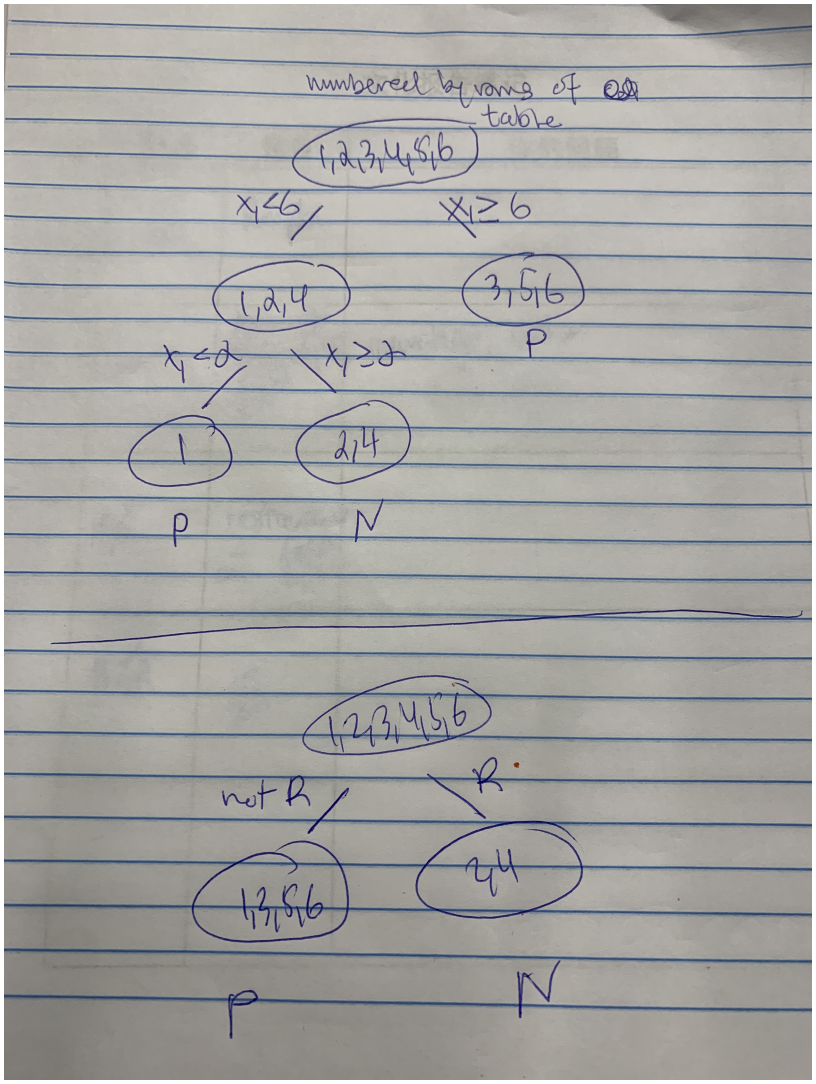

Figure 1: Problem 1

2. See the tree above.
3. For the categorical feature, we need to split along equality of the feature. The right thing to do is to split along red. See the second tree above.

Problem 2: Bagging

(a) Bagging or bootstrap aggregation is a powerful technique taken from statistics that is often used in Machine Learning. The process of bagging includes making m draws from a population $\mathcal{P}$ with replacement, training your classifier on the samples, and averaging out the results.

The expected squared error of such an ensemble classifier is

$$\mathbf{E}_{D_i}\left[\mathbf{E}_{(x,y)\sim\mathcal{P}}\left[(\hat{h}(x)-y)^2\right]\right],$$

where this first expected value is taken over all the randomness in the bagging process and the training of all the (otherwise identically trained) classifiers h_{D_i}, and the second expected value is over the selection of a random test point (x, y) selected from the source distribution $\mathcal{P}$. Prove that increasing the size of the ensemble (i.e. increasing m) cannot increase this expected squared error. (**Hint:** how does this affect the variance? the bias? the noise?)

(b) One way to estimate the prediction error would be to fit the model in question on a set of bootstrap samples and then keep track of how well it predicts the original training set.

If y^*_{ib} is the predicted value at of the ith training example x_i, from the model fitted to the bth bootstrap sample, our estimate is given by.

$$\mathbf{Err_{boot}} = \frac{1}{B}\frac{1}{N}\sum_{b=1}^{B}\sum_{i=1}^{N}\mathbf{L}(y_i, y^*_{ib})$$

Now consider that you have trained a bagged ensemble of 1-NN classifiers on a two-class classification problem, with the same number of observations in either class. Assume that the predictors and the class labels are independent. Show that the expectation of $\mathbf{Err_{boot}}$ does not do a good job of estimating the true error. Explain why this might be the case.

Hints:

1. Remember that the 1-NN classifier inherently has 0 training error.
2. You might also find it useful to remember

$$\lim_{N\to\infty}(1+\frac{x}{N})^N \approx e^x$$

c) Your friend suggests an alternative method of estimating the predictive error. She suggests using averages of classifiers in the ensemble that were trained on datasets that did not include the point (x_i, y_i) for each i in the training set. She claims that by computing the error of all these classifiers, we obtain an estimate of the true test error. Is this true? Can you show that this technique produces an unbiased estimate of test error?

Solution:

1. We can decompose the error into three components, bias variance and noise. We observe in the lecture note that bagging always reduces variance, so it is enough to show that the bias and noise do not increase upon increasing m. Now we can observe that the bias and noise is not affected by the number of classifiers we have, since the bias is a property of the model we are using. The noise is also not affected by the number of classifiers we have, since the noise is a property of the data we are using.

   Thus it is enough to show that the variance is nonincreasing. Given the datasets, they are independent and so the covariance is zero. Also note that in the lecture notes it says that bagging reduces variance. Therefore, increasing the number of classifiers cannot increase the expected squared error.

2. The true error is .5 because we are thinking about binary classifier. Therefore we need to compare the quantities $P(X \text{ not in train}) \cdot P(\text{wrong}|X \text{ not in train})$. We want to see that this quantity is less than .5. The second probablity is equal to .5. The first probability is equal to $(1 - \frac{1}{N})^N$ which becomes e^{-1}. Therefore the product is equal to $\frac{1}{2}e^{-1} < \frac{1}{2}$.

3. The approach is similar as before, we show that the quantity $P(X \text{ not in train}) \cdot P(\text{wrong}|X \text{ not in train})$ is equal .5 so we do get an estimate of the true test error. The second probability is equal to $\frac{1}{2}$. Again we reason about the first probability and then we see that it is equal to 1 because we are told that such is precisely the case in the problem statement.

Problem 3: Boosting

Background: In the Adaboost algorithm, recall that all weak learners h are binary, i.e. $h(\mathbf{x}_i) \in \{+1, -1\}$, and $y_i \in \{+1, -1\}$. Further, we let

$$w_t^{(i)} = e^{-y_i h_t(\mathbf{x}_i)} \Big/ \sum_i e^{-y_i h_t(\mathbf{x}_i)},$$

which means $\sum_{i=1}^n w_t^{(i)} = 1$. In iteration t, the algorithm updates:

$$w_{t+1}^{(i)} \leftarrow \frac{w_t^{(i)} \cdot e^{-\alpha_{t+1} h_{t+1}(\mathbf{x}_i) y_i}}{2\sqrt{\epsilon_{t+1}(1 - \epsilon_{t+1})}}. \tag{2}$$

where $\alpha_{t+1} = \frac{1}{2} \log\left(\frac{1-\epsilon_{t+1}}{\epsilon_{t+1}}\right)$ and $\epsilon_{t+1} = \sum_{i: h_{t+1}(\mathbf{x}_i) \neq y_i} w_t^{(i)}$.

Task: Prove that if $\sum_{i=1}^n w_t^{(i)} = 1$, and the $w_i^{(t)}$'s are updated with Equation (2), then $\sum_{i=1}^n w_{t+1}^{(i)} = 1$.

Hint: Show that

$$\sum_{i=1}^n w_t^{(i)} \cdot e^{-\alpha_{t+1} h_{t+1}(\mathbf{x}_i) y_i} = 2\sqrt{\epsilon_{t+1}(1 - \epsilon_{t+1})}.$$

Solution: We expand

$$\begin{aligned}\sum_i w_t^i \exp(-\alpha_{t+1} \cdot h_{t+1}(x_i) y_i) &= \frac{1}{\sum_i \exp(-y_i \cdot h_t(x_i))} \sum_i \exp(-y_i \cdot h_t(x_i)) \exp(-\alpha_{t+1} \cdot h_{t+1}(x_i) y_i) \\ &= \frac{1}{\sum_i \exp(-y_i \cdot h_t(x_i))} \sum_i \exp(-y_i \cdot h_t(x_i) - \alpha_{t+1} \cdot h_{t+1}(x_i) y_i)\end{aligned}$$

Therefore we see that

$$\sum_i w_t^i \exp(-\alpha_{t+1} \cdot h_{t+1}(x_i) y_i) = 2\sqrt{\epsilon_{t+1}(1 - \epsilon_{t+1})}$$

as desired.